

Political Alpha Tracker — Complete Project Documentation

A Quantamental Trading System that exploits the information asymmetry between political donations and government contract awards in the Indian stock market.

Table of Contents

- 0. [Prerequisites — What You Need to Know](#)
- 1. [The Core Thesis](#)
- 2. [Theoretical Foundation](#)
- 3. [System Architecture](#)
- 4. [Data Pipeline Overview](#)
- 5. [Module-by-Module Breakdown](#)
- 6. [The Knowledge Graph](#)
- 7. [The Conviction Scoring Engine](#)
- 8. [Technical Analysis Layer](#)
- 9. [Paper Trading Module](#)
- 10. [Backtesting Framework](#)
- 11. [The Streamlit Dashboard](#)
- 12. [Telegram Alert System](#)
- 13. [Deployment Architecture](#)
- 14. [Configuration Reference](#)
- 15. [Backtest Results](#)
- 16. [Deep Dive: Entity Resolution & Fuzzy Matching](#)
- 17. [Deep Dive: NLP & Large Language Models](#)

0. Prerequisites — What You Need to Know

If you want to build this project from scratch, here is everything you need — the domain knowledge, the technical skills, the accounts, and the data sources. Think of this as the "syllabus" behind the

system.

0.1 Domain Knowledge (The "Why")

You don't need to be an expert in all of these, but you need to understand the basics of each area. These are the intellectual building blocks behind every design decision in the system.

Indian Capital Markets

Topic	What to Learn	Why It Matters
How BSE/NSE work	Scrip codes, ISIN, corporate announcements, trading hours, settlement cycles	The entire data pipeline starts with BSE announcements
Market Cap categories	Large-cap (>₹20K Cr), Mid-cap (₹5K-20K Cr), Small-cap (<₹5K Cr), Micro-cap (<₹500 Cr)	Our alpha signal is strongest in small/micro-caps where analyst coverage is low
Corporate Actions	Board meetings, contract wins, buybacks, pledges, bulk/block deals, SAST disclosures	Each of these is a signal in our conviction scoring engine
India VIX	Volatility index derived from NIFTY options, measures market fear	We use VIX > 22 as a hard filter to avoid buying in panic markets
Transaction Costs	STT, stamp duty, GST, DP charges, SEBI turnover charges	Required for realistic paper trading simulation

Recommended Reading: Zerodha Varsity modules on [Indian Stock Markets](#), particularly the modules on fundamental analysis and trading.

Indian Political Economy

Topic	What to Learn	Why It Matters
Electoral Bonds	How they worked (2018-2024), SBI disclosure, Supreme Court ruling of Feb 2024	The single most important dataset — maps companies to political parties

Topic	What to Learn	Why It Matters
Electoral Trusts	Prudent Electoral Trust, Satya Electoral Trust, BJP Electoral Trust, etc.	The "middlemen" between donor companies and political parties
Government Procurement	GeM (Government e-Marketplace), CPPP, state tenders, L1 bidding process	Understanding how government contracts are awarded
State vs Central Politics	Which party rules which state, coalition dynamics	Required for the Regional Party Matching filter
MCA / ROC Filings	CIN (Corporate Identity Number), DIN (Director Identification Number), how company directors are registered	The mechanism for linking companies through shared directors

Recommended Reading:

- The SBI Electoral Bond disclosure dataset (available on the Election Commission of India website)
- ADR (Association for Democratic Reforms) reports on political funding
- Any good Indian political economy textbook or podcast (e.g., "The Seen and the Unseen")

Quantitative Finance

Topic	What to Learn	Why It Matters
Event Study Methodology	CAR (Cumulative Abnormal Returns), event windows, benchmark comparison	Used in our backtester to measure if the signal produces excess returns
Kelly Criterion	Optimal bet sizing formula: $f^* = p - q/b$, fractional Kelly	Used for position sizing in the paper trading module
Technical Analysis	RSI, MACD, SMA crossovers, OBV, ATR, VWAP	The entry-timing layer of the quantamental engine
Walk-Forward Optimization	TimeSeriesSplit, preventing overfitting in financial ML	Used in the XGBoost ML backtest to avoid data snooping
Factor Models	Multi-factor scoring, composite signals, hard filters vs soft scoring	The architecture of our conviction scoring engine

Recommended Reading:

- *Quantitative Trading* by Ernest Chan (the Kelly Criterion and backtesting chapters)
- *Advances in Financial Machine Learning* by Marcos López de Prado (walk-forward optimization, feature importance)
- Investopedia articles on RSI, MACD, OBV, ATR

Graph Theory / Network Science

Topic	What to Learn	Why It Matters
Directed Graphs	Nodes, edges, adjacency, path traversal	The knowledge graph is a NetworkX DiGraph
Graph Centrality	Degree centrality, betweenness centrality	Conceptual basis for our "exclusivity" scoring weight
Knowledge Graphs	Entity-Relationship modeling, node types, edge types	Our graph has 6 node types and 4 edge types

Recommended Reading: NetworkX documentation tutorials, any graph theory primer.

0.2 Technical Skills (The "How")

Programming Languages & Libraries

Skill	Level Needed	Used For
Python 3.10+	Intermediate-Advanced	The entire system is Python
SQL (SQLite)	Intermediate	All data storage, dashboard queries, caching
pandas	Intermediate	Data manipulation, CSV/Excel parsing, financial data wrangling
numpy	Basic	Numerical computations in technical analysis
NetworkX	Intermediate	Building and querying the knowledge graph
requests + BeautifulSoup	Intermediate	Web scraping BSE, MCA, Zaubacorp, PIB, GeM

Skill	Level Needed	Used For
Regular Expressions (regex)	Intermediate	Pattern matching on BSE announcement titles
yfinance	Basic	Fetching historical price data, financial statements
Plotly	Basic	Interactive charting in the Streamlit dashboard
Streamlit	Basic-Intermediate	Building the web dashboard
XGBoost + scikit-learn	Basic	ML optimization layer in the backtester

APIs & Services

Service	What You Need	How to Get It
Google Gemini API	API key for the Gemini LLM	Sign up at ai.google.dev , free tier is sufficient
Telegram Bot API	Bot token + Chat ID	Create a bot via @BotFather on Telegram
BSE India API	No key needed (public, unofficial)	Reverse-engineered REST endpoints, no registration required
yfinance	No key needed	Uses Yahoo Finance's public API under the hood

DevOps & Deployment

Skill	Level Needed	Used For
Git / GitHub	Basic	Version control, pushing code to remote
Docker	Basic	Containerizing the application
Linux CLI	Basic	Server deployment, cron jobs, systemd services
AWS EC2 (or any VPS)	Basic	Running the pipeline 24/7 in the cloud
cron	Basic	Scheduling the daily pipeline runs

0.3 Accounts & API Keys You Need

Before writing a single line of code, you need to set up the following:

1. **Google Gemini API Key** — Free tier at ai.google.dev. Used for NLP contract value extraction and competitor identification.
2. **Telegram Bot** — Create via [@BotFather](https://t.me/BotFather). You'll get a Bot Token. Then message the bot and use the `/getUpdates` API to find your Chat ID.
3. **GitHub Account** — For version control and CI/CD (optional but recommended).
4. **AWS Free Tier Account** (or any VPS) — For 24/7 cloud deployment. A `t2.micro` instance is sufficient.

These go into your `.env` file:

```
TELEGRAM_BOT_TOKEN=your_bot_token_here
TELEGRAM_CHAT_ID=your_chat_id_here
GEMINI_API_KEY=your_gemini_api_key_here
```

0.4 Data Sources You Need to Acquire

Data Source	Format	How to Get It	Used By
Electoral Bond Purchase Data	CSV	Download from ECI website (SBI disclosure)	<code>donor_ingester.py</code>
Electoral Bond Encashment Data	CSV	Download from ECI website (SBI disclosure)	<code>donor_ingester.py</code>
NSE Industry Mapping	CSV	Download from NSE India website (Nifty 500 classification)	<code>watchlist_generator.py</code>
BSE Corporate Announcements	JSON API	Live via BSE India API (no download needed)	<code>bse_monitor.py</code>
Company Director Data	HTML Scrape	Live via MCA / Zaubacorp (no download needed)	<code>mca_resolver.py</code>

Data Source	Format	How to Get It	Used By
Stock Price Data	API	Live via yfinance (no download needed)	financial_screener.py , technical_analyzer.py
India VIX	API	Live via yfinance ticker ^INDIAVIX	alpha_engine.py

The two Electoral Bond CSV files are the **critical seed data** — without them, there is no donor→party mapping, and the entire graph collapses.

0.5 Development Environment Setup

```
# 1. Clone the repository
git clone https://github.com/KalWardin-Dronzer/Political_Alpha_Tracker.git
cd Political_Alpha_Tracker

# 2. Create a virtual environment
python -m venv .venv

# 3. Activate it
# Windows:
.venv\Scripts\activate
# Linux/Mac:
source .venv/bin/activate

# 4. Install dependencies
pip install -r requirements.txt

# 5. Create your .env file (copy the example and fill in your keys)
cp .env.example .env
# Edit .env with your Telegram and Gemini API keys

# 6. Place your Electoral Bond CSVs in the data/ directory
# data/PurchaseData.csv
# data/EncashmentData.csv

# 7. Run the initial data refresh (builds the watchlist + graph)
python refresh.py

# 8. Run the daily pipeline (dry run first to verify)
python main.py --dry-run

# 9. Launch the dashboard
streamlit run app.py
```

0.6 Recommended Learning Path

If you're starting completely from scratch, here's the order I'd recommend:

Step	What to Learn	Time Estimate
1	Python fundamentals + pandas + SQL	2-4 weeks
2	How Indian stock markets work (Zerodha Varsity)	1 week
3	Web scraping with requests + BeautifulSoup	3-5 days
4	Graph theory basics + NetworkX	3-5 days
5	Read the Electoral Bond disclosure data, understand the structure	1-2 days
6	Technical Analysis basics (RSI, MACD, SMA)	3-5 days
7	Event Study methodology + backtesting concepts	3-5 days
8	Streamlit dashboard building	2-3 days
9	Docker + AWS deployment basics	2-3 days
10	Build the system module by module	2-4 weeks

Total estimated time: 2-3 months for someone with basic Python knowledge.

1. The Core Thesis

The Question

"If a company donates ₹50 crore to a political party through an Electoral Trust, and then wins a ₹500 crore government contract 6 months later — is that coincidence, or is it a tradeable alpha signal?"

The Hypothesis

In India, there exists a strong but largely hidden correlation between **political donations** (via Electoral Trusts and Electoral Bonds) and **government contract awards**. Companies that donate to the ruling party — whether at the central or state level — disproportionately win government contracts. This creates an information asymmetry that can be systematically exploited.

Why This Works (The Edge)

1. **Public Data, Private Insight:** All the data we use is publicly available (BSE announcements, Electoral Bond disclosures, MCA director filings). But *nobody is connecting these dots systematically*. The alpha comes from the **synthesis**, not the data itself.
2. **Structural Information Asymmetry:** Retail investors see a BSE announcement saying "XYZ Ltd wins ₹200 Cr order from Government of Maharashtra." They don't know that XYZ's director also sits on the board of a company that donated ₹30 Cr to the ruling party in Maharashtra. We do.
3. **The Indian Market is Inefficient for Micro/Small Caps:** Large-cap stocks are covered by 40+ analysts. But the micro-cap and small-cap defence/infra companies where this signal is strongest have minimal analyst coverage. The market takes weeks to fully price in the information.

2. Theoretical Foundation

2.1 Political Economy of Contracts

India's government procurement system is enormous. The Central and State governments award contracts worth **trillions of rupees** annually across defence, infrastructure, railways, power, and IT. The companies that win these contracts tend to be:

- In **government-dependent sectors**: Defence, Railways, Power, Road Construction, Water Treatment, Smart Meters
- **Small-to-mid cap** companies where a single ₹500 Cr contract can be 20%+ of their market cap (highly material)
- Companies with **board-level political connections** — directors who also sit on boards of major political donor companies

2.2 Electoral Trusts & Bonds

Electoral Trusts are registered entities that collect corporate donations and distribute them to political parties. Before 2024, **Electoral Bonds** were anonymous bearer instruments used for the same purpose. The Supreme Court of India struck down Electoral Bonds in February 2024 and ordered the SBI to disclose all purchasers.

This disclosure is the single most important dataset in our system. It tells us:

- Which company bought bonds
- How much they paid

- Which party encashed them
- When the transactions happened

2.3 The Graph Model (Network Theory)

We model the political-corporate ecosystem as a **directed graph** (a knowledge graph):

ListedCompany —SITS_ON_BOARD→ Director —SITS_ON_BOARD→ DonorCompany —DONATED_TO→ Elector

The **Alpha Query** traverses this graph. Given a company's CIN (Corporate Identity Number), it finds all paths to political parties and scores them based on:

- **Exclusivity** (40% weight): If a director sits on only 2 boards (the listed company and the donor), that's a tight, exclusive connection. If they sit on 15 boards, the connection is diluted.
- **Proximity** (30% weight): Fewer hops = stronger signal. A direct Company→Director→Donor→Trust path (3 hops) is stronger than one with intermediary companies.
- **Magnitude** (30% weight): A ₹100 Cr donation is a stronger signal than a ₹10 Lakh donation.

2.4 Quantamental Approach

We use a **"Quantamental"** methodology — a hybrid of:

- **Quantitative**: Graph algorithms, statistical backtesting, XGBoost ML models, technical indicators
- **Fundamental**: Balance sheet screening (Debt/Equity, Cash Flow, Promoter Pledge)

The weighting is approximately **70-80% Fundamentals/Macro** and **20-30% Technicals** (the "when to enter" layer).

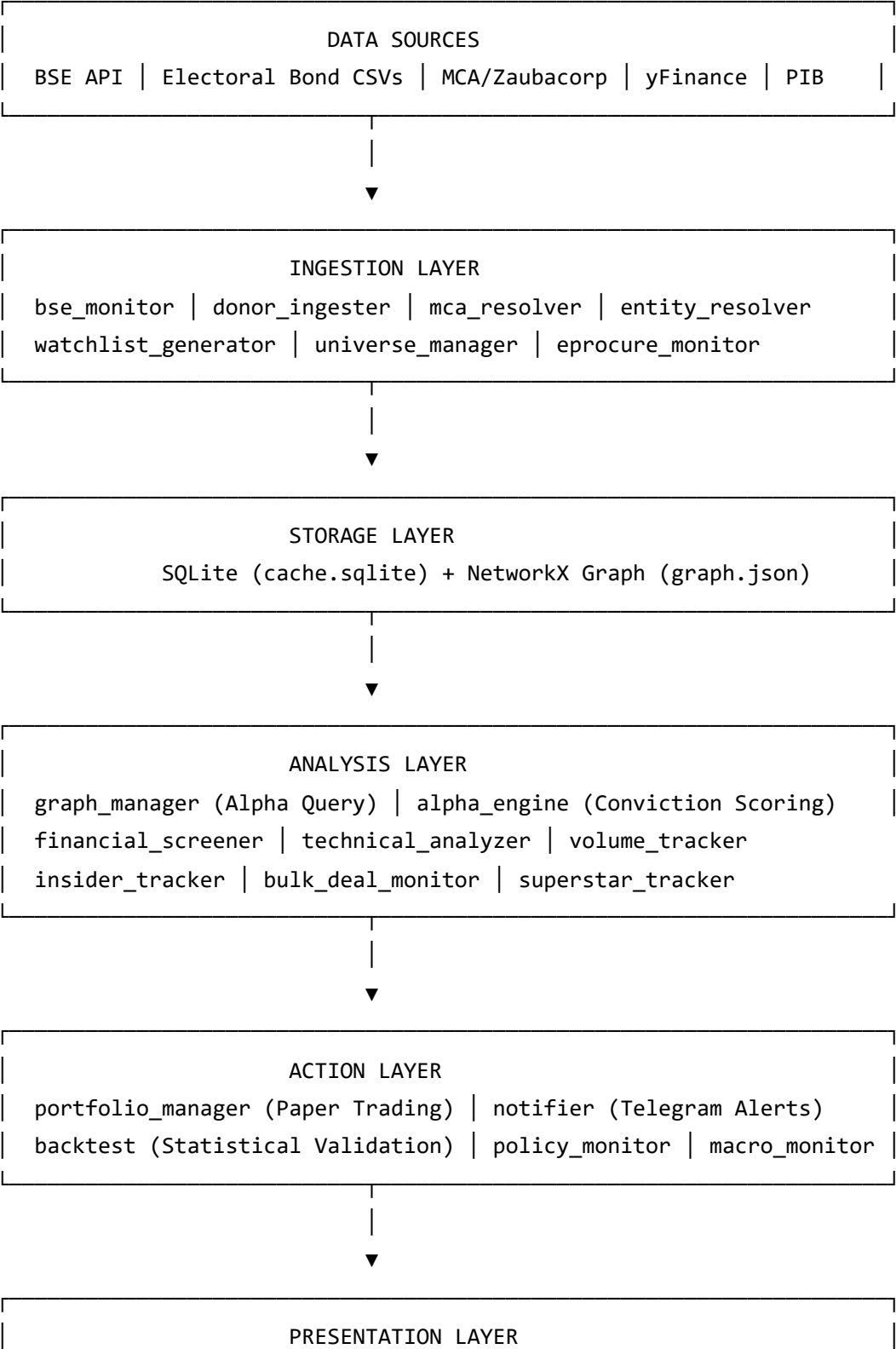
2.5 Key Academic Concepts Used

Concept	Application
Network Centrality	Scoring political connections via graph traversal
Event Study Methodology (CAR)	Measuring excess returns around contract announcements
Kelly Criterion	Optimal position sizing for paper trades
Walk-Forward Optimization	Preventing overfitting in the ML backtest

Concept	Application
RSI / MACD / OBV	Technical entry timing (momentum, trend, volume confirmation)
ATR Trailing Stop	Dynamic risk management for the paper portfolio

3. System Architecture

High-Level Architecture Diagram



Technology Stack

Component	Technology
Language	Python 3.11
Database	SQLite (via <code>sqlite3</code>)
Graph Engine	NetworkX
Price Data	yfinance
NLP / LLM	Google Gemini API
Web Scraping	requests + BeautifulSoup
Fuzzy Matching	RapidFuzz
ML	XGBoost + scikit-learn
Dashboard	Streamlit + Plotly
Alerts	Telegram Bot API
Deployment	AWS EC2 + cron

4. Data Pipeline Overview

The system runs a **daily automated pipeline** via `main.py` → `PipelineOrchestrator` . Here's the exact execution sequence:

Step 1: Poll Telegram for `/exit` commands

The system checks if the user has sent `/exit` `SCRIP_CODE` via Telegram to manually close a paper position.

Step 2: Build Monitoring Universe

`universe_manager.py` compiles the full list of scrip codes to monitor. This includes:

- The 33-company private watchlist (generated quarterly by `watchlist_generator.py`)
- Any additional universe expansion companies

Step 3: Scan BSE Announcements

`bse_monitor.py` calls the BSE India API for each scrip code, fetches the last day's corporate announcements, and classifies them:

- **Contract Events:** Regex-matched against patterns like "Award of Order", "Work Order Received", "LOA", etc.
- **Board Changes:** "Appointment of Director", "Resignation", etc.

Step 3.5: Execute Virtual Sells

The `paper_trader.execute_sells()` method reviews all open paper positions. It sells any position where:

- The current price has dropped **below the ATR trailing stop-loss**, or
- The position has been held for **>90 days** without materializing

Step 3.6: Scan Bulk/Block Deals

`bulk_deal_monitor.py` checks if any of the tracked "superstar investors" (Ashish Kacholia, Dolly Khanna, Vijay Kedia, etc.) have made bulk/block purchases in watchlist stocks.

Step 4: Process Board Changes

If a director appointment/resignation is detected, the system refreshes that company's director data from MCA/Zaubacorp and rebuilds the graph.

Step 4.5: Check eProcure L1 Bids

`eprocure_monitor.py` checks the government eProcurement portal for L1 (lowest) bidders. This is an **early alpha signal** — if a company is declared L1 bidder on a tender, the formal contract award BSE announcement typically follows 2-4 weeks later.

Step 5: Process Contract Events (The Core Logic)

For each detected contract announcement:

1. **Fundamental Gate:** `financial_screener.py` checks Debt/Equity, Cash Flow, and Promoter Pledge. Companies that fail are rejected.
2. **CIN Lookup:** The company's CIN is retrieved from the database.
3. **Director Refresh:** If the director data is stale (>90 days), it's refreshed from MCA.
4. **Graph Rebuild:** The knowledge graph is rebuilt with latest data.
5. **Alpha Query:** The graph is traversed to find political connections and compute the alpha score.
6. **Conviction Scoring:** The `AlphaEngine` computes a multi-factor Conviction Score (0-13.5).
7. **Technical Analysis:** The `TechnicalAnalyzer` runs RSI, MACD, OBV, SMA, ATR on the stock's price data.
8. **Alert Decision:** If Conviction Score $\geq 4.0 \rightarrow$ fire a Telegram alert and execute a paper buy.

Step 5.5: Policy & Macro Monitoring

- `policy_monitor.py` scans PIB (Press Information Bureau) for government policy announcements that could create sector tailwinds.
- `macro_event_monitor.py` checks for global macro events (oil price shocks, Fed rate decisions, India-specific catalysts).

Step 5.6: Advanced Alpha Scans

- `tender_monitor.py` : Scans GeM/CPPP government tender portals.
- `state_budget_monitor.py` : Monitors state budget allocations for sector-level capex boosts.
- `pledge_monitor.py` : Tracks promoter pledge changes (a red flag signal).

Step 6: Wrap Up

Save the graph, send a daily summary to Telegram, and log the run.

5. Module-by-Module Breakdown

src/config.py — Central Configuration

All constants, thresholds, API URLs, regex patterns, and tunable parameters live here. Nothing is hardcoded in modules. Key sections:

- **Paths:** Database, graph file, CSV locations
- **Telegram:** Bot token and chat ID (from .env)
- **Sector Keywords:** The list of government-dependent sectors used to build the watchlist
- **BSE Regex:** Patterns to detect contract wins and board changes from announcement titles
- **Fundamental Thresholds:** Max D/E ratio (5.0), Max Promoter Pledge (50%), Market Cap range (₹50 Cr – ₹10L Cr)
- **Alpha Scoring Weights:** Exclusivity (40%), Proximity (30%), Magnitude (30%)
- **Election Cycle:** Upcoming election dates with a 1.5x multiplier for pre-election alpha boost
- **State-Party Mapping:** Maps Indian states to their ruling parties for regional contract matching

src/cache_manager.py — SQLite Database Layer

The single source of truth. All modules read/write through CacheManager . Tables:

Table	Purpose
companies	BSE-listed companies with CIN, sector, market cap, watchlist flag
directors	Company directors with DIN, resolved from MCA/Zaubacorp
donors	Electoral Trust / Bond donor records (amount, party, year)
announcements	BSE corporate announcements (cached, with contract flag)
alpha_graph	Computed alpha scores per DIN-CIN pair
virtual_portfolio	Open paper trading positions
trade_history	Closed paper trades with full P&L and tax breakdown
held_positions	Active high-conviction alerts (auto-expiring)
pledges	Promoter pledge tracking data
bulk_deals	Bulk/block deal records

Table	Purpose
superstar_holdings	Superstar investor shareholding snapshots
tenders	Government tender records
system_log	Pipeline health tracking

src/watchlist_generator.py — Watchlist Construction

Builds the target watchlist through a **five-stage funnel**:

1. **Stage A1 — Sector Sweep**: Fetches the NSE equity listing and filters companies in government-dependent sectors (defence, railways, power, infrastructure, etc.)
2. **Stage A2 — Donor Match**: Cross-references Electoral Bond donors against listed companies using fuzzy name matching (RapidFuzz, threshold 75%). Companies with ₹10 Cr+ donations are auto-included.
3. **Stage B — Market Cap Filter**: Keeps companies with market cap between ₹50 Cr and ₹10,00,000 Cr.
4. **Stage C — Contract Frequency**: Ranks remaining companies by the number of BSE contract announcements in the last year. Companies with zero contracts are dropped.
5. **Stage D — Fundamental Gate**: Runs the Financial Screener to discard companies with dangerous balance sheets.

Output: ~33 companies, refreshed quarterly.

src/bse_monitor.py — BSE Announcement Scanner

Monitors the BSE India corporate announcements API. For each scrip code:

1. Calls the BSE India API endpoint with the scrip code and date range
2. Parses the JSON response for announcement titles
3. Applies regex pattern matching to classify each announcement as:
 - contract (order wins, LOAs, tenders)
 - board_change (director appointments/resignations)
4. Excludes false positives (NCLT orders, SEBI orders, etc.)
5. Calls the AlphaEngine to extract materiality data (contract value as % of market cap)

src/mca_resolver.py — Director Data Resolution

Resolves company directors from the Ministry of Corporate Affairs (MCA) or Zaubacorp:

1. Takes a CIN (Corporate Identity Number)
2. Scrapes director names and DINs (Director Identification Numbers)
3. Stores them in the `directors` table
4. Identifies ex-bureaucrats using designation heuristics

`src/entity_resolver.py` — Fuzzy Entity Matching

Solves the critical problem of matching entities across different data sources:

- Electoral Bond says "Megha Engineering & Infrastructures Ltd"
- BSE says "Megha Engineering and Infrastructures Limited"
- MCA says "MEGHA ENGINEERING & INFRASTRUCTURES LTD."

Uses RapidFuzz with a configurable threshold to match these variations.

`src/donor_ingester.py` — Electoral Data Ingestion

Ingests political donation data from multiple sources:

1. **Electoral Bond CSVs** (SBI disclosure) — Purchase and Encashment data
2. **ECI Contribution Reports** (Excel files from Election Commission)
3. **MyNeta HTML Tables** (scrape-friendly electoral trust data)
4. **ADR PDF Reports** (last resort, uses tabula-py)

`src/graph_manager.py` — Knowledge Graph Engine

The heart of the system. Maintains a NetworkX directed graph with:

Node Types:

- `ListedCompany` — BSE-listed companies (identified by CIN)
- `Director` — Company directors (identified by DIN)
- `DonorCompany` — Companies that donated to electoral trusts/bonds
- `ElectoralTrust` — Registered electoral trusts
- `PoliticalParty` — Political parties that received funds
- `Tender` — Government contracts won by companies

Edge Types:

- `SITS_ON_BOARD` — `Director` → `Company`
- `DONATED_TO` — `DonorCompany` → `ElectoralTrust`

- FUNDED — ElectoralTrust → PoliticalParty
- WON_CONTRACT — ListedCompany → Tender

The Alpha Query Algorithm:

Given: A company's CIN

1. Find the company node
2. Find all directors on its board
3. For each director, find all OTHER companies they sit on
4. For each of those companies, check if they donated to an Electoral Trust
5. If yes, trace the path to the Political Party
6. Score the connection:
 - Exclusivity = $1 / (\text{number of boards the shared director sits on})$
 - Proximity = $1 / (\text{number of hops in the path})$
 - Magnitude = $\log(\text{donation_amount}) / \log(\text{max_donation})$
 - Alpha Score = $0.4 * \text{Exclusivity} + 0.3 * \text{Proximity} + 0.3 * \text{Magnitude}$
7. Apply Election Cycle Multiplier (1.5x if election within 12 months)
8. Return sorted list of connections with scores

src/financial_screener.py — The Porinju Layer

Named after Porinju Veliyath (a famous Indian value investor), this module filters out fundamentally unhealthy companies:

1. **Debt-to-Equity Ratio:** Must be < 5.0 (loosened from the typical 2.0 to accommodate capital-intensive infra companies)
2. **Operating Cash Flow:** Must be non-negative at least once in the last 2 years
3. **Promoter Pledge:** Must be $< 50\%$ (high pledge = promoter is desperate for cash)
4. **Market Cap Range:** ₹50 Cr – ₹10,00,000 Cr

Data is fetched from **yfinance** (BSE suffix `.BO`, NSE suffix `.NS`).

src/alpha_engine.py — Conviction Scoring & NLP

The central scoring and NLP engine. Two main functions:

1. `parse_contract_details()` : Uses the Gemini LLM to extract the exact ₹ value and issuing state from BSE announcement text. Falls back to regex parsing if API is unavailable.
2. `calculate_conviction_score()` : See Section 7 below.
3. `find_competitors()` : Uses Gemini to identify unconnected competitors for pair-trading ideas.

src/technical_analyzer.py — Technical Analysis Engine

See Section 8 below.

src/portfolio_manager.py — Paper Trading Engine

See Section 9 below.

src/volume_tracker.py — Smart Money Volume Detection

Computes a Z-Score on rolling 20-day average volume. If today's volume is >2.5 standard deviations above the mean, it flags a "volume spike" — potential front-running by informed traders before a contract announcement.

src/insider_tracker.py — Insider Transaction Monitoring

Detects **cluster buys** — when 3+ insiders buy shares within the same 30-day window. Also detects SAST (Substantial Acquisition of Shares and Takeovers) filings where an external acquirer is building a position.

src/bulk_deal_monitor.py — Bulk/Block Deal Scanner

Monitors BSE/NSE bulk and block deal data for purchases by tracked "superstar" investors.

src/superstar_tracker.py — Superstar Investor Tracking

Tracks new entries by famous Indian investors (Ashish Kacholia, Dolly Khanna, Vijay Kedia, etc.) into watchlist stocks via quarterly shareholding disclosures.

src/tender_monitor.py — Government Tender Scanner

Monitors the Government e-Marketplace (GeM) and CPPP (Central Public Procurement Portal) for new tenders relevant to watchlist companies.

src/state_budget_monitor.py — State Budget Capex Tracker

Monitors state budget announcements for sector-level capital expenditure increases. If a state announces a 40% increase in road construction spending, all road construction companies in the watchlist get a boost.

`src/pledge_monitor.py` — Promoter Pledge Tracker

Tracks changes in promoter pledge levels. A sudden increase in pledging is a **red flag** (the promoter is leveraging their shares for personal loans — a sign of financial stress).

`src/policy_monitor.py` — PIB Policy Scanner

Scans the Press Information Bureau for policy announcements that could create sector tailwinds (e.g., "Government approves ₹1.5 Lakh Crore for Railway electrification").

`src/macro_event_monitor.py` — Global Macro Event Monitor

Checks for global events (oil price shocks, FOMC rate decisions, India-Pakistan tensions) and assesses impact on watchlist companies.

`src/bureaucrat_resolver.py` — Ex-Bureaucrat Detection

Identifies directors who are former IAS/IPS officers or retired government bureaucrats. These "revolving door" appointments are a strong signal of political connections.

`src/universe_manager.py` — Universe Expansion

Manages the broader monitoring universe beyond the core watchlist. Can expand to include companies mentioned in policy announcements or identified through competitor analysis.

`src/eprocure_monitor.py` — eProcurement L1 Bid Scanner

Checks government eProcurement portals for L1 (lowest) bidder declarations. This is an **early signal** — the formal BSE announcement follows weeks later.

6. The Knowledge Graph

What It Contains

As of the last refresh, the graph contains:

- **~132 nodes**: Companies, Directors, Donors, Electoral Trusts, Political Parties
- **~171 edges**: Board seats, donations, contracts

How It's Built

1. **Company Nodes**: Added when the watchlist is generated
2. **Director Nodes**: Added when MCA/Zaubacorp resolves a company's board
3. **Donor Nodes**: Added when Electoral Bond/Trust data is ingested
4. **Party Nodes**: Added from the encashment side of Electoral Bond data
5. **Tender Nodes**: Added when a contract announcement is detected

Visualization

The graph is visualized using `pyvis` (a NetworkX → interactive HTML converter). Running `visualize.py` generates `graph_visualization.html` which can be opened in any browser. The Streamlit dashboard embeds this HTML in the "Knowledge Graph" tab.

Graph Pruning

To prevent the graph from growing unbounded:

- Tenders older than **24 months** are removed
- Donor records older than **5 years** are removed

7. The Conviction Scoring Engine

The Conviction Score is a **multi-factor composite score** ranging from 0 to 13.5. It determines whether a detected contract event is worth trading on.

Hard Filters (Instant Rejection)

Before scoring begins, three hard filters can reject a signal outright (score = 0):

1. **Regional Party Mismatch**: If the contract is from a state where the donation recipient party is NOT in power, the signal is rejected. (e.g., donating to BJP but winning a contract from a TMC-ruled West Bengal)
2. **High VIX Regime**: If India VIX > 22 (high market fear), no new positions are taken.
3. **High Promoter Pledge**: If promoter pledge > 25%, the company is too risky.

Scoring Factors (8 Factors)

#	Factor	Max Points	Source
1	Contract Materiality	+2.0	If contract value ≥ 5% of market cap
2	Corporate Buyback	+1.5	If buyback ≥ 2% of market cap
3	Political Connection	+0.5	If Alpha Query returns a score > 0
4	Insider Cluster Buying	+2.0	If 3+ insiders bought in last 30 days
5	SAST External Acquirer	+2.0	If a hostile/whale entity is accumulating
6	Smart Money Bulk Deal	+1.5	If a tracked investor made a bulk purchase
7	Superstar New Entry	+1.0	If a famous investor entered the stock
8	Technical Analysis	+2.5 / -1.5	Based on TA signal (STRONG_BUY to AVOID)

Maximum Possible Score: 13.5 (all factors firing)

Alert Threshold

- **Score ≥ 4.0:** Fire a Telegram alert + execute a paper buy
- **Score < 4.0:** Log the event but don't alert

Position Sizing (Kelly Criterion)

Conviction Score	Allocation %	Kelly Tier
≥ 8.0	25% of capital	Full Kelly cap
≥ 6.0	15% of capital	Half Kelly cap
≥ 4.0	5% of capital	Quarter Kelly cap
< 4.0	0%	No trade

8. Technical Analysis Layer

The TA layer answers **"When to enter?"** — it doesn't override fundamental decisions but adjusts the timing and conviction level.

Indicator Stack (5 Non-Redundant Indicators)

Indicator	Category	Why It's Included
RSI (14-period)	Momentum	Detects oversold entry opportunities and overbought warnings
MACD (12, 26, 9)	Momentum/Trend	Confirms trend direction via crossovers
50/200-day SMA	Trend	Golden/Death cross for long-term trend confirmation
OBV (On-Balance Volume)	Volume	Confirms price moves with volume (smart money accumulation/distribution)
VWAP (20-day rolling)	Fair Value	Institutional entry/exit reference price

Plus **ATR (14-period)** for dynamic stop-loss calculation (not scored, used for risk management).

Scoring Logic (0-10 Scale)

Component	Scoring Rule	Max Points
RSI	Oversold (<30) = +2.0, Favorable (30-40) = +1.5, Neutral (40-55) = +1.0, Warm (55-70) = +0.5, Overbought (>70) = 0	2.0
MACD	Bullish crossover/expansion = +2.0, Positive histogram = +1.0, Bearish = 0	2.0
SMA Crossover	Golden Cross + Price > SMA50 = +2.0, Golden Cross but price below = +1.0, Death Cross = 0	2.0
OBV Slope	10-day upslope (accumulation) = +2.0, Downslope (distribution) = 0	2.0
VWAP	Price > VWAP = +2.0, Below = 0	2.0

Signal Classification

Score Range	Signal	Conviction Adjustment
8-10	STRONG_BUY	+2.5 points
6-7.9	BUY	+1.0 point
3-5.9	NEUTRAL	0.0 points
0-2.9	AVOID	-1.5 points

ATR Trailing Stop-Loss

The system calculates a **2x ATR trailing stop** for every position:

$$\text{ATR Stop} = \text{Current Price} - (2 \times \text{ATR}_{14})$$

If BEL is trading at ₹401 and $\text{ATR}(14) = ₹7.05$, the stop-loss is $₹401 - (2 \times 7.05) = ₹386.91$.

Point-in-Time Simulation

For backtesting, the TA engine accepts an `end_date` parameter. When provided, it fetches price data **ending on that date** (1 year lookback), computing the indicators as they would have appeared on that historical date. This eliminates **look-ahead bias**.

9. Paper Trading Module

Overview

Since we're not deploying real capital (student project), the `PaperTrader` class simulates realistic equity delivery trades on the Indian market.

Realistic Cost Model (Zerodha Pricing)

Every paper trade accounts for real-world transaction costs:

Cost Component	Rate
Brokerage	₹0 (Zerodha delivery)
STT (Securities Transaction Tax)	0.1% on buy + sell
NSE Transaction Charges	0.00325%
SEBI Charges	₹10 per crore
Stamp Duty	0.015% (buy only)
GST	18% on (brokerage + txn + SEBI)
DP Charges	₹15.93 flat per sell
Slippage	0.5% (accounts for illiquid micro-cap spreads)

Buy Logic

1. Check if the stock is already in the portfolio (no duplicate positions)
2. Calculate available cash (initial capital + realized P&L - invested amount)
3. Determine position size using the Kelly Criterion tiers (see Section 7)
4. Apply slippage (0.5% worse execution price)
5. Calculate exact taxes and fees
6. Record the trade in `virtual_portfolio`

Sell Logic (Two Triggers)

1. **ATR Stop-Loss:** If the current price drops below the 2x ATR trailing stop, the position is sold immediately (cutting losses).
2. **Time-Stop:** If the position has been held for ≥ 90 days without hitting the stop-loss, it's sold (the thesis has either played out or failed).

Initial Capital

The paper portfolio starts with **₹1,00,000** (one lakh rupees).

10. Backtesting Framework

The `Backtester` class runs four statistical tests to validate the alpha signal:

Test 1: Base Rate of Connectivity

Question: What percentage of *random* micro-caps also show political connections?

Method: Randomly selects 30 non-watchlist companies in the same market cap range and runs the Alpha Query on them.

Pass Criteria: Control group connection rate < 50%. If random companies are just as connected, the signal is noise.

Result: Watchlist 26.3% vs Control 8.3% → **Signal is meaningful** ✓

Test 2: Post-Event Excess Returns

Question: Do politically-connected contract winners outperform the benchmark?

Method: For every historical contract announcement, fetch the stock's forward returns at 30/60/90/180/360 days. Compare against the NIFTY index return over the same period.

Key Result: +13.92% excess alpha at 180 days ✓

Test 3: Win Rate (Hit Ratio)

Question: What percentage of alerts would have been profitable?

Method: Simulate every historical event through the full conviction scoring engine (including point-in-time Technical Analysis). Count how many produced positive 90-day returns.

Pass Criteria: Win rate ≥ 55% (to cover transaction costs)

Result: 66.7% win rate (6/9 trades profitable) ✓

Test 4: ML Optimization (XGBoost)

Question: Can a machine learning model improve the signal?

Method: Train an XGBoost classifier with heavy regularization (max_depth=2, L1+L2) using Walk-Forward cross-validation (TimeSeriesSplit). Features: alpha_score, materiality_pct, volume_z_score,

election_multiplier.

Result: 66.67% out-of-sample accuracy. Volume Z-Score is the most important feature (confirming the "smart money" hypothesis) 

Overall Verdict: SIGNAL VALIDATED (ML APPROVED)

11. The Streamlit Dashboard

The dashboard (`app.py`) provides a real-time visual interface with six tabs:

Tab 1: Overview

- KPI cards: Active Positions, Realized P&L, Material Tenders Parsed
- Historical backtest performance bar chart
- Live Alpha Alerts table
- Recent Corporate Announcements feed

Tab 2: Conviction Rankings

- All companies ranked by their alpha graph score
- Styled with gradient color mapping

Tab 3: Knowledge Graph

- Embedded interactive `pyvis` HTML visualization
- Nodes colored by type (companies, directors, donors, parties)
- Clickable, draggable, zoomable

Tab 4: Paper Portfolio

- Virtual portfolio performance metrics
- Current Holdings table
- Trade History with color-coded P&L (green = profit, red = loss)

Tab 5: Technical Analysis

- Company selector dropdown

- Signal banner with color-coded styling
- KPI metrics (CMP, RSI, SMA 50, SMA 200, ATR Stop)
- Interactive 4-panel Plotly chart:
 - Row 1: Candlestick + SMA 50/200 + VWAP + ATR Stop
 - Row 2: RSI with overbought/oversold bands
 - Row 3: MACD line + signal + histogram
 - Row 4: On-Balance Volume area chart

Tab 6: Chat with Data

- Natural language query interface powered by Gemini
- Auto-generates and executes SQL queries against the SQLite database
- Example: "What are the top 3 companies by market cap?"

12. Telegram Alert System

Alert Types

1. **Alpha Alert** (High Priority): Fired when Conviction Score ≥ 4.0
 - Company name, scrip code, announcement title
 - Political connection details (director, donor, party)
 - Fundamental health summary (D/E, Cash Flow, Pledge)
 - Conviction Score breakdown (all 8 factors)
 - Technical Analysis summary (RSI, MACD, Trend, OBV, ATR Stop)
 - Competitor list for pair trading
2. **Volume Spike Alert**: When a watchlist stock shows $>2.5\sigma$ volume anomaly
3. **Policy Alert**: When a PIB policy benefits a watchlist sector
4. **Macro Event Alert**: When a global event impacts watchlist companies
5. **System Alert**: Pipeline errors, empty watchlist warnings
6. **Daily Summary**: End-of-day report with contracts found, alerts fired, graph stats

Position Management via Telegram

- Send `/exit SCRIP_CODE` to manually close a paper position
- The system polls for these commands at the start of each pipeline run

13. Deployment Architecture

AWS EC2 Setup

The system is deployed on an AWS EC2 instance with:

- `main.py` running via **cron job** (daily at 6:00 PM IST, after market close)
- `main.py --scan-volume` running separately (daily at 3:30 PM IST, near market close)
- `streamlit run app.py` running as a persistent `systemd` service on port 8501
- Docker and docker-compose available for containerized deployment

Cron Schedule (from `crontab_setup.txt`)

```
# Daily pipeline (6:00 PM IST = 12:30 UTC)
30 12 * * 1-5 cd /path/to/project && python main.py >> data/pipeline.log 2>&1

# Volume scan (3:30 PM IST = 10:00 UTC)
0 10 * * 1-5 cd /path/to/project && python main.py --scan-volume >> data/volume.log 2>&1
```

Environment Variables (`.env`)

```
TELEGRAM_BOT_TOKEN=your_bot_token
TELEGRAM_CHAT_ID=your_chat_id
GEMINI_API_KEY=your_gemini_api_key
```

14. Configuration Reference

All tunable parameters in `src/config.py` :

Fundamental Thresholds

Parameter	Default	Description
MAX_DEBT_TO_EQUITY	5.0	Maximum D/E ratio to pass screening
MAX_PROMOTER_PLEDGE_PCT	50.0	Maximum promoter pledge %

Parameter	Default	Description
MARKET_CAP_MIN_CR	50	Minimum market cap (₹ Crore)
MARKET_CAP_MAX_CR	1,000,000	Maximum market cap (₹ Crore)

Alpha Scoring

Parameter	Default	Description
ALPHA_WEIGHT_EXCLUSIVITY	0.4	Weight for director board exclusivity
ALPHA_WEIGHT_PROXIMITY	0.3	Weight for path length proximity
ALPHA_WEIGHT_MAGNITUDE	0.3	Weight for donation size
ALPHA_SCORE_THRESHOLD	0.5	Minimum score to consider a connection valid
MAX_PATH_HOPS	3	Maximum graph traversal depth

Election Cycle

Parameter	Default	Description
ELECTION_MULTIPLIER	1.5	Alpha score boost within 12 months of an election

Paper Trading

Parameter	Default	Description
initial_capital	₹1,00,000	Starting paper trading capital
slippage_pct	0.5%	Assumed execution slippage
max_hold_days	90	Maximum holding period before time-stop

15. Backtest Results

Final Validated Results (August 2026)

Test	Result	Status
Base Rate of Connectivity	Watchlist 26.3% vs Control 8.3%	✓ Meaningful
180-Day Excess Return	+13.92% vs Benchmark	✓ Outperformance
Win Rate (Conviction ≥ 4.0)	66.7% (6/9 trades)	✓ Viable
ML Out-of-Sample Accuracy	66.67%	✓ Validated
Overall Verdict	SIGNAL VALIDATED (ML APPROVED)	✓

Key Insight

The system's edge comes from the **combination** of factors:

- 1. Political connection gives us the **"what"** (which company will benefit)
- 2. Fundamental screening gives us the **"safety"** (don't buy junk)
- 3. Technical analysis gives us the **"when"** (enter on strength, avoid falling knives)
- 4. The conviction scoring synthesizes everything into a single actionable number

No single factor alone produces a viable win rate. It's the multi-factor synthesis — the quantamental stack — that creates the edge.

File Structure

```
Political_Alpha_Tracker/
├─ main.py                # CLI entry point (daily pipeline)
├─ app.py                 # Streamlit dashboard
├─ refresh.py             # Quarterly watchlist refresh
├─ visualize.py           # Graph → HTML visualization
├─ run_backtest.py        # Backtest runner
├─ requirements.txt       # Python dependencies
├─ Dockerfile             # Container definition
├─ docker-compose.yml     # Multi-service orchestration
├─ aws_deploy.sh          # AWS deployment script
├─ crontab_setup.txt      # Cron job schedule
├─ .env                   # Secrets (not committed)
├─ .env.example           # Template for secrets
├─
├─ src/
│   ├─ config.py          # Central configuration
│   ├─ cache_manager.py   # SQLite database layer
│   ├─ pipeline_orchestrator.py # Main pipeline logic
│   ├─ bse_monitor.py     # BSE announcement scanner
│   ├─ alpha_engine.py    # Conviction scoring + NLP
│   ├─ graph_manager.py   # Knowledge graph (NetworkX)
│   ├─ financial_screener.py # Fundamental health checks
│   ├─ technical_analyzer.py # RSI/MACD/OBV/SMA/ATR engine
│   ├─ portfolio_manager.py # Paper trading engine
│   ├─ backtest.py        # Statistical validation
│   ├─ watchlist_generator.py # Automatic watchlist construction
│   ├─ universe_manager.py # Universe expansion
│   ├─ donor_ingester.py  # Electoral data ingestion
│   ├─ mca_resolver.py    # MCA director resolution
│   ├─ entity_resolver.py # Fuzzy entity matching
│   ├─ notifier.py        # Telegram alerts
│   ├─ volume_tracker.py  # Smart money volume detection
│   ├─ insider_tracker.py # Insider transaction monitoring
│   ├─ bulk_deal_monitor.py # Bulk/block deal scanner
│   ├─ superstar_tracker.py # Superstar investor tracker
│   ├─ tender_monitor.py  # GeM/CPPP tender scanner
│   ├─ state_budget_monitor.py # State budget capex tracker
│   ├─ pledge_monitor.py  # Promoter pledge tracker
│   ├─ policy_monitor.py  # PIB policy scanner
│   └─ macro_event_monitor.py # Global macro event monitor
```

```
| | — bureaucrat_resolver.py # Ex-bureaucrat detection
| | — eprocure_monitor.py   # L1 bid scanner
|
| — data/
| | — cache.sqlite          # Main database
| | — graph.json            # Serialized knowledge graph
| | — PurchaseData.csv      # Electoral Bond purchases (SBI disclosure)
| | — EncashmentData.csv    # Electoral Bond encashments
| | — pipeline.log          # Pipeline execution logs
|
| — assets/
| | — style.css             # Dashboard custom styles
|
| — tests/                  # Test scripts
```

16. Deep Dive: Entity Resolution & Fuzzy Matching

If you know nothing about data engineering, read this first.

Imagine you have two lists of people.

List A (from a school database) says: "John F. Kennedy"

List B (from a hospital) says: "Kennedy, John F"

A human knows these are the same person. But to a computer, the text string "John F. Kennedy" is completely different from "Kennedy, John F". If a computer tries to join these two lists using an exact match (`ListA.Name == ListB.Name`), it will fail and say there is no match.

This exact problem happens in our project, but with **Company Names**.

We have data from two very different sources:

1. **The Electoral Bond Data (SBI):** Names are messy, typed by bank clerks. Example:

MEGHA ENGINEERING LTD

2. **The Stock Market Data (BSE):** Names are official and formal. Example:

Megha Engineering & Infrastructures Limited

If we can't link these two names together, our entire Knowledge Graph breaks. We wouldn't know that the company trading on the stock market is the exact same company that bought the political bonds.

The Solution: Fuzzy Matching

"Fuzzy matching" is a technique that calculates *how similar* two strings of text are, usually giving a score from 0 to 100.

In this project, we use a Python library called **RapidFuzz**. RapidFuzz uses an algorithm called the *Levenshtein Distance*.

The Levenshtein Distance counts the minimum number of single-character edits (insertions, deletions, or substitutions) required to change one word into the other.

For example, to change "kitten" to "sitting":

1. **s**itten (substitution of "s" for "k")
2. sitt**i**n (substitution of "i" for "e")
3. sitt**ing** (insertion of "g" at the end)

It took 3 edits.

How Our Code Does It (`src/entity_resolver.py`)

We don't just throw raw names at the fuzzy matcher. If we compare "Reliance Industries Limited" with "Reliance Industries", the word "Limited" will lower the similarity score, even though it's the exact same company.

Step 1: Aggressive Cleaning (Standardization)

Our code first runs the company names through a cleaner that strips away all corporate suffixes. It removes words like: LTD , LIMITED , PVT , PRIVATE , INC , CORP , LLC .

It also removes all punctuation and makes everything uppercase.

So:

- Reliance Industries Limited -> RELIANCE INDUSTRIES
- Reliance Ind. Ltd. -> RELIANCE IND

Step 2: Token Set Ratio

RapidFuzz has different ways of scoring. We use `fuzz.token_set_ratio` .

Instead of comparing the whole string at once, this method breaks the string into individual words (tokens), sorts them alphabetically, and then compares them.

This is incredibly powerful because it ignores word order.

`fuzz.token_set_ratio("ENGINEERING MEGHA", "MEGHA ENGINEERING")` will return a perfect 100 score.

Step 3: Thresholding

If the RapidFuzz score is above our threshold (defined as `DONOR_MATCH_SCORE = 75` in our `config.py`), the system accepts it as a match and links the Electoral Bond donor to the BSE listed company in our database.

17. Deep Dive: NLP & Large Language Models

If you know nothing about AI/LLMs, read this first.

Every day, the Bombay Stock Exchange (BSE) publishes hundreds of corporate announcements. When a company wins a new government contract, they upload a PDF document announcing it.

These PDFs are written for humans, not computers. They contain paragraphs like:

"We are pleased to inform you that our company has emerged as the Lowest Bidder (L1) for a project awarded by the Maharashtra State Road Development Corporation. The total estimated value of the contract is Rs. 500.5 Crores."

Our trading system needs exactly two pieces of information from this text:

1. How much is the contract worth? (500.5)
2. Which state government gave the contract? (Maharashtra)

We cannot use simple rules (like "find the number next to 'Rs'") because every company formats their PDFs differently. Some say `Rs. 500.5 Cr`, some say `INR 5,000,000,000`, some say `Five Hundred Crores`.

The Solution: Large Language Models (LLMs)

A Large Language Model (like OpenAI's ChatGPT or Google's Gemini) is an AI that has read the entire internet and understands human language.

Instead of writing complex rules to find the money value, we can literally just hand the text to the AI and ask it a question.

In this project, we use the **Google Gemini API** (via the `google.generativeai` library) in `src/alpha_engine.py`.

How Our Code Does It

Step 1: Extracting Text from PDFs

When the `bse_monitor.py` detects a "contract award" announcement, it downloads the PDF file.

Our code then uses a library called `pypdf` to read the actual text out of the first 3 pages of the PDF.

Step 2: The Prompt (Instructing the AI)

We send the extracted text to the Gemini API, along with a very strict set of instructions called a "Prompt".

Our prompt looks something like this:

"You are a financial data extraction bot. Read the following corporate announcement. Find the total monetary value of the contract awarded, and convert it to Crores. Also find the name of the state government that awarded it. You must reply ONLY in JSON format, like this: {"contract_value_cr": 500.5, "issuing_authority_state": "maharashtra"} "

Step 3: JSON Parsing

Because we instructed the AI to reply in JSON (JavaScript Object Notation), it gives us the data in a format that Python can easily read as a dictionary.

```
data = {  
    "contract_value_cr": 500.5,  
    "issuing_authority_state": "maharashtra"  
}
```

Now, our Python code knows exactly how much the contract is worth.

What if the AI fails? (The Fallback)

APIs can sometimes go down, or you might run out of free credits.

Because this is a critical trading system, we built a **Regex Fallback**.

Regex (Regular Expressions) is a traditional way to search for patterns in text.

If the Gemini API fails, our code falls back to running this regex pattern:

```
r"(?i)rs\.\s*(\d+(?:\.\d+)?)\s*(?:cr|crore)"
```

This pattern tells the computer: *Find the letters "rs", followed by optional spaces, then capture any numbers (including decimals), followed by the word "cr" or "crore".*

It's not as smart as the AI, and it won't catch every edge case, but it ensures the pipeline doesn't completely crash if the AI is unavailable.

This documentation was generated on August 10, 2026. The Political Alpha Tracker is an academic research project exploring the intersection of political economy, network theory, and quantitative

finance in the Indian equity market.